

Powell (DFP) algorithm (sometimes referred to just as the *Fletcher-Powell* method) or the closely related *Broyden-Fletcher-Goldfarb-Shanno (BFGS)* algorithm. These methods require of order N^2 storage, derivative calculations, and one-dimensional subminimization. Details are in §10.9. Our personal experience is that the quasi-Newton methods dominate the conjugate gradient methods (if you can afford the storage), but there are probably applications where the reverse is true.

Finally, the case where the method uses gradients, but you are willing to let them be calculated by extra function evaluations (and finite differences):

- In our experience, the quasi-Newton (variable metric) methods work very well in this case, so much so that they can be significantly more efficient than Powell's method on suitable problems. In §10.9 we give an implementation that (almost) hides the gradient calculation completely. Thus quasi-Newton is effectively our first choice of method both when you are willing to calculate gradients, and when you are not willing!

You can now proceed to scale the peaks (and/or plumb the depths) of practical optimization.

CITED REFERENCES AND FURTHER READING:

- Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).
- Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press).
- Gill, P.E., Murray, W., and Wright, M.H. 1981, *Practical Optimization* (New York: Academic Press).
- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 17.
- Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1.
- Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover).
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Chapter 10.

10.1 Initially Bracketing a Minimum

What does it mean to *bracket* a minimum? A root of a function is known to be bracketed by a pair of points, a and b , when the function has opposite sign at those two points. A minimum, by contrast, is known to be bracketed only when there is a *triplet* of points, $a < b < c$ (or $c < b < a$), such that $f(b)$ is less than both $f(a)$ and $f(c)$. In this case we know that the function (if it is smooth) has a minimum in the interval (a, c) .

We consider the initial bracketing of a minimum to be an essential part of any one-dimensional minimization. There are some one-dimensional algorithms that do not require a rigorous initial bracketing. However, we would *never* trade the secure feeling of *knowing* that a minimum is “in there somewhere” for the dubious reduction of function evaluations that these nonbracketing routines may promise. Please bracket your minima (or, for that matter, your zeros) before isolating them!

There is not much theory as to how to do this bracketing. Obviously you want to step downhill. But how far? We like to take larger and larger steps, starting with some (wild?) initial guess and then increasing the stepsize at each step either by a constant factor, or else by the result of a parabolic extrapolation of the preceding points that is designed to take us to the extrapolated turning point. It doesn't much matter if the steps get big. After all, we are stepping downhill, so we already have the left and middle points of the bracketing triplet. We just need to take a big enough step to stop the downhill trend and get a high third point.

Here is our standard routine, the function `bracket`. It appears in the structure `Bracketmethod` that serves as the base class for all the one-dimensional minimization methods we give in this chapter.

`struct Bracketmethod {`

`mins.h`

Base class for one-dimensional minimization routines. Provides a routine to bracket a minimum and several utility functions.

`Doub ax,bx,cx,fa,fb,fc;`

`template <class T>`

`void bracket(const Doub a, const Doub b, T &func)`

Given a function or functor `func`, and given distinct initial points `ax` and `bx`, this routine searches in the downhill direction (defined by the function as evaluated at the initial points) and returns new points `ax`, `bx`, `cx` that bracket a minimum of the function. Also returned are the function values at the three points, `fa`, `fb`, and `fc`.

`{`

`const Doub GOLD=1.618034, GLIMIT=100.0, TINY=1.0e-20;`

Here `GOLD` is the default ratio by which successive intervals are magnified and `GLIMIT` is the maximum magnification allowed for a parabolic-fit step.

`ax=a; bx=b;`

`Doub fu;`

`fa=func(ax);`

`fb=func(bx);`

`if (fb > fa) {`

`SWAP(ax,bx);`

`SWAP(fb,fa);`

`}`

Switch roles of *a* and *b* so that we can go downhill in the direction from *a* to *b*.

`cx=bx+GOLD*(bx-ax);`

First guess for *c*.

`fc=func(cx);`

`while (fb > fc) {`

Keep returning here until we bracket.

`Doub r=(bx-ax)*(fb-fc);`

Compute *u* by parabolic extrapolation from *a*, *b*, *c*. `TINY` is used to prevent any possible division by zero.

`Doub q=(bx-cx)*(fb-fa);`

`Doub u=bx-((bx-cx)*q-(bx-ax)*r)/`
`(2.0*SIGN(MAX(abs(q-r),TINY),q-r));`

`Doub ulim=bx+GLIMIT*(cx-bx);`

We won't go farther than this. Test various possibilities:

`if ((bx-u)*(u-cx) > 0.0) {`

Parabolic *u* is between *b* and *c*: try it.

`fu=func(u);`

`if (fu < fc) {`

Got a minimum between *b* and *c*.

`ax=bx;`

`bx=u;`

`fa=fb;`

`fb=fu;`

`return;`

`} else if (fu > fb) {`

Got a minimum between *a* and *u*.

`cx=u;`

`fc=fu;`

`return;`

`}`

`u=cx+GOLD*(cx-bx);`

Parabolic fit was no use. Use default magnification.

`fu=func(u);`

`} else if ((cx-u)*(u-ulim) > 0.0) {`

Parabolic fit is between *c* and its allowed limit.

`fu=func(u);`